Project Report

OPEN-SOURCE SOFTWARE
(LINUX ADMINISTRATION AUDIT)

Name: Dhruv Chaudhary

Registration Number: 24BEY10002

Chosen Software: Git

Introduction

It is pretty much impossible to find any software today that doesn't use open source at some level. Most of the apps and tools we use daily are built on it. Unlike the closed-off software made by big companies, open source is totally transparent. You can actually see the code, change it, and share it. That kind of freedom is the main reason it’s so popular—it just makes it easier for developers to work together without a bunch of restrictions.

I chose Git for this project because it's the one tool every developer needs. It is easily the most used version control system out there. If you have a team working on the same files, Git is what stops the whole thing from turning into a mess. It tracks every change and keeps everything organized so nothing gets lost.

The goal of this audit is to look at Git from a few different angles. I'll be covering where it came from, its license, and why it is so important to the dev community. I also put together some shell scripts to show how these tools actually run on a Linux system.

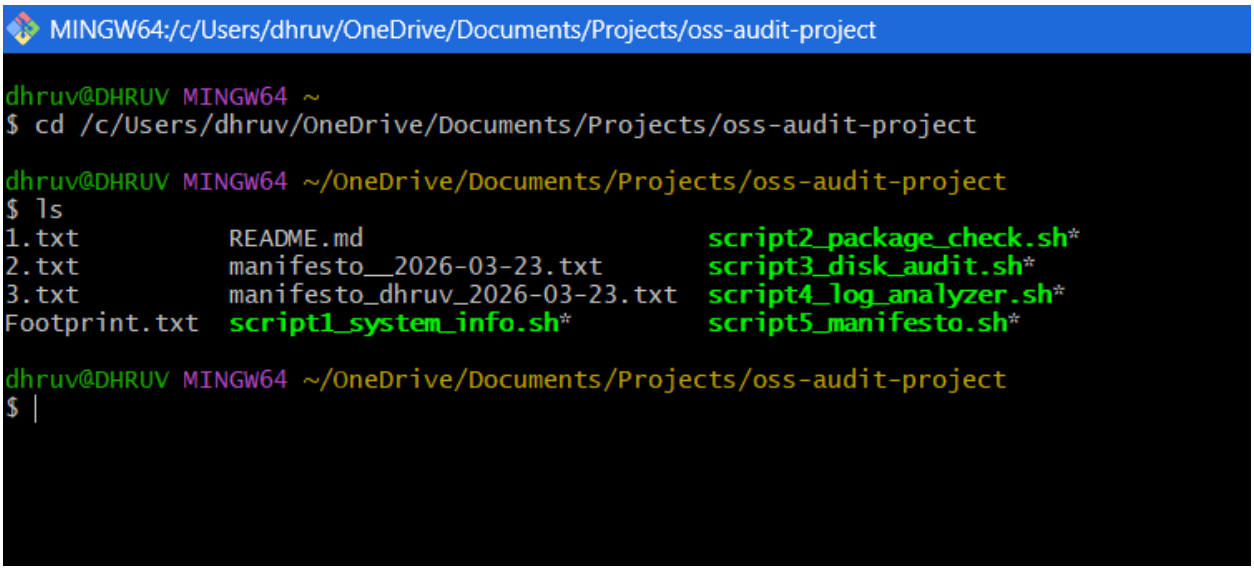

Figure 1: Project folder showing all scripts and files

Part A

Origin and Philosophy

A1) Origin of Git

Before Git showed up, trying to manage something as massive as the Linux kernel was a total nightmare. Since developers were scattered all over the place, they basically had to use this tool called BitKeeper just to keep track of their changes. The catch was that BitKeeper wasn't open source—it was private software—so the devs couldn't actually see how it worked under the hood. Even so, the Linux team didn't have much of a choice because it was pretty much the only thing back then that didn't crash when handling a project that big.

Then, everything basically fell apart. There was a big disagreement between the Linux community and the company that owned BitKeeper. The developers ended up losing their access entirely. It was a disaster for the project. Without a way to track code, trying to merge changes and keep things organized became nearly impossible. Everything just stalled.

Linus Torvalds, who ran the Linux project, had been frustrated enough facing this issue over the time. He was just over the whole idea of having to rely on some company’s private software ever again. So, back in 2005, frustrated him basically just sat down and built his own system from the scratch and that is exactly how Git got started. He made sure the entire thing was fast, decentralized, and most importantly, totally open source so that nobody could ever pull it away from the community again.

Git was designed by keeping few clear goals in mind. It needed to be fast, reliable, and should be able to handle really large projects without slowing down things. One of its main features is that it is well and widely distributed, which basically means every developer has their own full copy of the project. This allows people to work independently without depending on a central server too much.

Another important feature about Git is that it is open source. This means that anyone can use it, look at how it works, and even modify it however they need it to work. Over time, Git became very popular and is now used in most software development environments.

A2) License Analysis

From what I understood, open-source software is not just about using a tool, it’s more about having some control over it. We are not completely dependent on the people who made it. If we want, we can actually go into the code, see how things work, and even make changes as per how we want. That’s something we don’t really get with normal software.

Because of this, people usually explain open-source using four basic freedom(s):

- The freedom to run the software for any purpose
- The freedom to study how the software works
- The freedom to modify the software
- The freedom to share the software

These freedom(s) are really important because they just make everything more open and easier to handle. Instead of being hidden, developers can actually see what’s happening with their code and learn from how others do the things. It saves a massive amount of time since people don't have to keep rebuilding the same stuff from scratch over and over.

Git specifically uses the GPL license. Basically, it is there to make sure these freedoms stay protected. It means anyone can just access the source code, change it however they want it to, and share those changes with everyone.

One of the biggest rules with this license is that if someone modifies Git and then gives it out to others, they have to share their new code as well. You can’t just take it, change a few things, and then lock it down as your own private software. That’s the main thing that stops Git from ever turning into a closed-source product.

A company can take Git, modify it, and even sell it if they want. But the catch is they can’t keep those changes private if they are sharing the software with others. They have to make their changes available too. That’s what helps keep Git open and stops it from turning into something closed.

The GPL license is seen as strict mainly because of this rule. It focuses on keeping software open for everyone. On the other hand, the MIT license has a lot more flexible rule. It lets developers use the code however they want—even for closed-source or commercial projects—without forcing them to share their changes back.

Personally, I would probably go with the MIT license for my own stuff. The main reason is that it’s just way more flexible to work with and gives you more freedom in how the code gets used. This is especially true in real-world situations where a company might want to keep their specific features private.

A3) Ethics of Open Source

Open source is not just about the software itself. It's more about how people work together around it. A lot of developers contribute on their own, and that slowly improves things.

One thing I like about it is that you can actually learn from real projects instead of just theory. It also makes things more accessible since most of it is available to everyone. But at the same time, not everything can be open source. Companies spend time and money building products, so they also need to earn from it. So, it makes more sense to have a mix instead of forcing everything to be open.

It is ethical for companies to use open-source software and still make profit. This is because companies usually add their own features, improvements, and services. This creates additional value beyond the original software. At the same time, companies should respect the open-source community.

They should follow the license rules and at least give credit to the original developers. If companies just use open-source work without giving anything back, it can feel a bit unfair.

The idea of "standing on the shoulders of giants" is just about building on what already exists instead of starting from scratch every time. In software, this is pretty normal. People use libraries, frameworks, and all sorts of tools that others have already made, and then build on top of that. Open source makes this easier since the code is already out there.

Part B

Linux Footprint

Installation

For this project, I used a Windows system where I downloaded Git from the official website and installed it normally. After that, I used Git Bash since it gives a Linux-like terminal on Windows and is easy to work on. This made it easier to run commands and test the scripts.

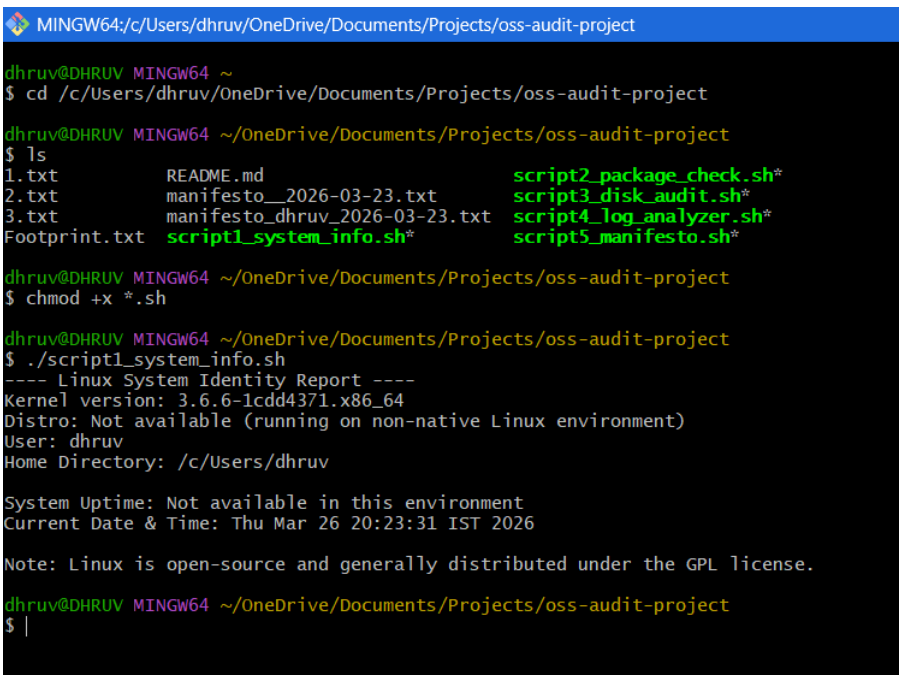

Figure 2: Output of Script 1 showing system information

File Locations

In Linux systems, files are placed in a fixed folder structure, and Git also follows that. The executable files are usually in /usr/bin. Config files can be found in /etc/gitconfig or sometimes in ~/.gitconfig for user settings. Logs are mostly in /var/log, but I didn't really see all of them since I was using Git Bash.

```
dhruv@DHRUV MINGW64 ~/OneDrive/Documents/Projects/oss-audit-project
$ ./script2_package_check.sh
Inspecting package: git
Git is the selected software for this audit.
Version info (fallback):
git version 2.53.0.windows.2
License: GPL-2.0
```

Figure 3: Script checking Git installation and version

User and Security

Git runs as a normal user and not root. That's actually better for safety. Running things as root can be risky because it has full access. Using a normal user keeps things limited so even if something goes wrong, it won't affect everything.

```
dhruv@DHRUV MINGW64 ~/OneDrive/Documents/Projects/oss-audit-project
$ ./script2_package_check.sh git
Inspecting package: git
Git is the selected software for this audit.
Version info (fallback):
git version 2.53.0.windows.2
License: GPL-2.0
```

Figure 4: Script using argument to inspect Git

```
dhruv@DHRUV MINGW64 ~/OneDrive/Documents/Projects/oss-audit-project
$ ./script3_disk_audit.sh
Starting disk and permission audit...

-> Directory: /etc
   Size: 1.6M
   Perms: drwxr-xr-x - Owner: dhruv
------------------------
Directory /var/log not found
------------------------
Directory /home not found
------------------------
-> Directory: /usr/bin
   Size: 91M
   Perms: drwxr-xr-x - Owner: dhruv
------------------------
-> Directory: /tmp
   Size: 1.5G
   Perms: drwxr-xr-x - Owner: dhruv
------------------------

Checking for Git config...
Found global git config at /etc/gitconfig
```

Figure 5: Disk usage and permission audit of directories

Service Management

Git is not really a service. It doesn't run in the background like Apache or MySQL. It only works when you use it through commands. So, there's nothing to start or stop here.

Updates

Git usually gets updated through tools like apt or dnf in Linux. When the system is updated, Git also updates if needed. So, it's not something you have to install again every time.

```
dhruv@DHRUV MINGW64 ~/OneDrive/Documents/Projects/oss-audit-project
$ ./script4_log_analyzer.sh script1_system_info.sh
Searching for 'error' inside script1_system_info.sh...

Total matches: 0
No matches found.

Log file size: 4.0K
```

Figure 6: Log analyzer script searching for keyword "error"

```
dhruv@DHRUV MINGW64 ~/OneDrive/Documents/Projects/oss-audit-project
$ ./script5_manifesto.sh
--- Open Source Manifesto Generator ---

Which tool do you use the most? Git
What does freedom mean to you in one word? Independence
What would you like to build in the future? AI assistant

Generating manifesto...

Saved to manifesto_dhruv_2026-03-26.txt

My Open Source Manifesto
------------------------
The tool I use the most is Git.
For me, freedom in open source means Independence.
In the future, I would like to build AI assistant.
Written by dhruv on 2026-03-26

dhruv@DHRUV MINGW64 ~/OneDrive/Documents/Projects/oss-audit-project
$ |
```

Figure 7: Interactive manifesto generation script output

Part C

FOSS Ecosystem

Git is written in C. It depends on system libraries and works closely with the file system. Some parts also use shell commands.

Over time it became a core tool in development.

Platforms like GitHub and GitLab are built around it.

They allow people to collaborate from different places.

Git is mainly used to track changes in code and helps manage versions.

Prevents confusion when multiple people are working together.

Not part of LAMP directly, but still used in most workflows.

There is also a large open-source community around Git.

People contribute, fix bugs, and improve it regularly.

Part D

Open-Source vs Proprietary

Feature	Git (Open Source)	BitKeeper (Paid)
Cost	Free	Paid
Security	Open for review	Closed source
Support	Community-based	Company-based
Flexibility	High	Limited
Control	Full user control	Company controlled

Conclusion

Git is an open-source tool with very flexible rules, and it has honestly changed the script on how developers get their work done. It just makes working together so much easier and keeps everything from becoming messy, while also being a massive part of why open-source works so well.

To be honest, it’s hard to even imagine writing code today without it. It completely changed the game for how teams actually collaborate and stay organized. The way it backs the whole open-source movement is probably its biggest win.

It is like night and day when you compare Git to those old-school proprietary tools. Git basically hands the keys back to the developers. Instead of struggling around with some private rules set by a company, you actually own your own workflow. Since everything is out in the open environment, there’s always someone or the other trying out to fix a random bug or to drop a new usable feature, so the whole thing stays fresh without getting stuck in the past.

At the end of the day, you just can't really get around using Git anymore. It doesn't matter if you're just messing with a tiny side project or working a massive corporate job—it is the standard. As open source keeps taking over, tools like Git are going to stay right at the heart of how everything gets built.